Vision-Based Target Tracking for a Quadrotor UAV

Model-Based and Learned Compensation of Camera Latency

How to run the simulations

Abbass Nasrallah

Master 2 — Robotics and Intelligent Systems

Lebanese University, Faculty of Engineering

MATLAB R2024a with Simulink

1. What this folder contains

Nine Simulink models and the MATLAB files they need. Each model is one condition of the comparison: no delay, the 160 ms delay left uncompensated, and the same delay with each of the four compensators in the loop. The plant, the camera, the three controllers and the delay block are written inside every model as MATLAB Function blocks, so the only external files are the compensators themselves and the plotting scripts.

```
New folder\
│
├── QuadVisionControl.slx                 no delay — the reference
├── QuadVisionControl_delay.slx           160 ms delay, uncompensated
├── QuadVisionControl_dkf.slx             delay + Delayed Kalman Filter
├── QuadVisionControl_luenberger.slx      delay + Delayed Luenberger observer
├── QuadVisionControl_lstm.slx            delay + LSTM residual predictor
├── QuadVisionControl_obs.slx             delay + Neural Luenberger Observer
├── QuadVisionControl_noise_delay.slx     noisy camera, uncompensated
├── QuadVisionControl_noise_dkf.slx       noisy camera + DKF
├── QuadVisionControl_noise_luen.slx      noisy camera + Luenberger
│
├── observers\
│   ├── dkf\                  dkf_block.m · dkf_core.m · dkf_init.m · dkf_params.m
│   ├── dkf+noise\            dkf_block_noise.m · dkf_core.m · dkf_init.m · dkf_params_noise.m
│   ├── luenberger\           luen_block.m · luen_core.m · luen_init.m · luen_params.m
│   ├── luenberger+noise\     luen_block_noise.m · luen_core.m · luen_init.m · luen_params_noise.m
│   ├── lstm\                 lstm_block.m · lstm_forward_core.m · lstm_weights.mat
│   │                          lstm_config.m · smooth_block.m
│   └── neural luenberger\    velocity_net_block.m · obs_integrator_block.m
│                              lstm_forward_core.m · lstm_weights.mat
│
└── plots\                    run_all_sims.m · pickrun.m · getsig.m · loadruns.m · runmany.m
                               plot_distance_error.m · compare_distance.m · compare_trajectory.m
                               compare_z_saturation.m · plot_z_saturation.m · visualize_camera.m
                               visualize_combined.m · visualize_3d_trajectory_old.m
                               visualize_3d_trajectory_spotlight.m
```

2. Setting up, once

Open MATLAB, then set `ROOT` to wherever this folder sits on your machine and run these four lines. Everything in this guide assumes they have been run.

```
ROOT = 'C:\Users\Abbass\Desktop\New folder';   % <-- change to your own path
cd(ROOT)
addpath(fullfile(ROOT, 'plots'))
savepath                                       % optional: remember it next time
```

The compensator folders are **not** added here. Each method gets its own folder added just before it is used, and the reason matters:

> **Why one observer folder at a time.** `observers\lstm\` and `observers\neural luenberger\` each contain a file named `lstm_weights.mat` — the same name, different trained networks. If both folders are on the MATLAB path at once, whichever comes first wins and the other model silently flies the wrong network. Add the folder you need, and remove the other one. Every recipe below does this on its first two lines.

3. The two things every run needs

3.1 The scenario number, `idx`

Every model reads a variable called `idx` from the MATLAB workspace. It chooses which way the target flies. If it is not defined the model will not start.

idx	Target motion	Velocity
1	Stationary	0
2	Vertical climb — reported as Scenario 1	Vz = 20 m/s
3	Move −X	Vx = −3 m/s
4	Move +Y	Vy = 3 m/s
5	Lateral and climbing — reported as Scenario 2	Vy = 5, Vz = 20
6	X + Y	Vx = 1, Vy = 1
7	X + Z	Vx = 1, Vz = 20
8	Fully three-dimensional — reported as Scenario 3	Vx = 1, Vy = 1, Vz = 20
9 … 15	Manoeuvring target: same vertical path, oscillating speed	Vz = 15 + A·sin(2πt/Tp)

The manoeuvring set holds the speed between 11 and 19 m/s, below the vehicle's 21 m/s vertical limit, and varies only the rhythm: idx 9, 10, 11 are Tp = 10 s with A = 1, 2, 4 (peak 0.63, 1.26, 2.51 m/s²); idx 14 is Tp = 6 s (4.19 m/s²); idx 15 is Tp = 5 s (5.03 m/s²); idx 12 is Tp = 4 s (6.28 m/s²); idx 13 is Tp = 2.5 s (10.05 m/s²). **idx 14 is the case that separates the two model-based estimators**: the DKF tracks it, the Luenberger observer loses the target.

3.2 The initial condition

Every run starts from the same state: 9.28 m from the target, at rest, level, yaw 2 rad. Two of the models — the LSTM and the Neural Luenberger — read that state from four workspace variables, so define them before running anything:

```
IC_pos   = [15; 5; 7];      % start position, m
IC_vel   = [0; 0; 0];       % at rest
IC_euler = [0; 0; 2];       % level, yaw = 2 rad
IC_omega = [0; 0; 0];       % no body rates
```

The other seven models have those numbers built in and ignore these variables, so it is safe — and simplest — to define them once at the start of the session and leave them alone.

> **Nothing else to configure.** Solver (`ode4`, fixed step 5 ms) and stop time (30 s) are saved inside every model. The camera delay is 160 ms, the frame rate 30 Hz, the commanded stand-off 2 m.

4. Running each model

Each recipe below is self-contained: copy the whole block into the Command Window and press Enter. The result is stored in `out1 … out8` or `out10`, which is the number the plotting scripts ask for. Run them in this order the first time; each one stands alone afterwards.

Change the scenario by editing the single `idx` line. The examples use `idx = 8`, the fully three-dimensional target, which is the hardest of the three reported cases and the one in the headline figures.

4.1 No delay — the reference

The loop with a perfect, instantaneous camera. This is the ceiling every other run is compared against. It needs no compensator files at all.

```
idx  = 8;
out1 = sim('QuadVisionControl');
```

4.2 160 ms delay, uncompensated — the floor

The same loop, the same controllers, the only change being the delay. On idx 5 and idx 8 it never reaches the stand-off inside 30 s. Also needs no extra files.

```
idx  = 8;
out2 = sim('QuadVisionControl_delay');
```

4.3 Delayed Kalman Filter

```
rmpath(fullfile(ROOT,'observers','dkf+noise'))      % in case it was on
addpath(fullfile(ROOT,'observers','dkf'))

clear dkf_block dkf_core dkf_init
idx  = 8;
out3 = sim('QuadVisionControl_dkf');
```

The filter is in `dkf_core.m`; its constants, including σ_a = [5 5 10] and σ_{uv} = 0.5 px, are in `dkf_params.m`. Nothing needs editing to reproduce the reported result.

4.4 Delayed Luenberger observer

```
rmpath(fullfile(ROOT,'observers','luenberger+noise'))
addpath(fullfile(ROOT,'observers','luenberger'))

clear luen_block luen_core luen_init
idx  = 8;
out4 = sim('QuadVisionControl_luenberger');
```

Same architecture as the DKF, fixed gains instead of a covariance. `luen_params.m` holds ω_n = 8 rad/s and ζ = 1, which give L_p = 0.413 and L_v = 1.644 s^{-1}.

4.5 LSTM residual predictor

```
rmpath(fullfile(ROOT,'observers','neural luenberger'))   % IMPORTANT
addpath(fullfile(ROOT,'observers','lstm'))

clear lstm_block lstm_forward_core     % REQUIRED: wipes the network's memory
IC_pos   = [15; 5; 7];
IC_vel   = [0; 0; 0];
IC_euler = [0; 0; 2];
IC_omega = [0; 0; 0];
idx  = 8;
out5 = sim('QuadVisionControl_lstm');
```

The weights already in the folder are the round-4 filter-aware network, the one reported as the LSTM result. The output low-pass is inside the model ($\tau = 0.02$ s); with $\tau = 0$ the same weights diverge, so leave it alone. `clear lstm_block lstm_forward_core` is not optional here: the network keeps its memory in persistent variables between simulations.

> **Switching the network.** Both learned methods load a file called `lstm_weights.mat`, and the correct one for each is already installed, so nothing needs switching to reproduce the reported results. To put a different trained network in place, copy it over that name and clear the memory — these two lines are what `use_model` does on the author's machine:
>
> ```
> L = fullfile(ROOT,'observers','lstm');
> copyfile(fullfile(L,'lstm_weights_r4f.mat'), fullfile(L,'lstm_weights.mat'))
> clear lstm_block lstm_forward_core
> ```
>
> The same pattern applies in `observers\neural luenberger\` with `lstm_weights_obs2.mat`. `use_model('r4f')` itself is not included: it reads its weight folder from an absolute path in `lstm_config.m` that only exists on the machine the project was built on.

4.6 Neural Luenberger Observer

```
rmpath(fullfile(ROOT,'observers','lstm'))               % IMPORTANT
addpath(fullfile(ROOT,'observers','neural luenberger'))

clear velocity_net_block obs_integrator_block lstm_forward_core   % REQUIRED
IC_pos   = [15; 5; 7];
IC_vel   = [0; 0; 0];
IC_euler = [0; 0; 2];
IC_omega = [0; 0; 0];
idx   = 8;
out10 = sim('QuadVisionControl_obs');
```

Note the run number: this one is **out10**, not out6. The network here predicts image-plane velocity and the observer integrates it, with the delayed innovation fed back at $L = 0.10$. The gain is baked into what the network learned, so changing `L_UV` or `L_R` in `obs_integrator_block.m` invalidates the weights.

> **The `clear` line is not optional.** `velocity_net_block`, `obs_integrator_block` and `lstm_forward_core` are ordinary MATLAB functions called from inside the model, and they keep the network's hidden state, the history buffer and the integrator in persistent variables. Those survive between simulations, so a second run without the clear starts with the previous flight still in memory. The same is true of `lstm_block` and `lstm_forward_core` in section 4.5.

4.7 The noise campaign

The same three conditions with a noisy detector: 1.5 px on the centroid and 0.25 px on the apparent radius, drawn once per frame from a fixed seed, so every method meets exactly the same noise sequence. The two compensators are re-

tuned for it — that is the point of the campaign.

```
% --- noisy baseline, no compensation ---
idx  = 8;
out8 = sim('QuadVisionControl_noise_delay');

% --- DKF under noise  (sigma_uv 1.5, sigma_a [2 2 4]) ---
rmpath(fullfile(ROOT,'observers','dkf'))
addpath(fullfile(ROOT,'observers','dkf+noise'))
clear dkf_block_noise dkf_core dkf_init
idx  = 8;
out6 = sim('QuadVisionControl_noise_dkf');

% --- Luenberger under noise  (omega_n retuned 8 -> 4) ---
rmpath(fullfile(ROOT,'observers','luenberger'))
addpath(fullfile(ROOT,'observers','luenberger+noise'))
clear luen_block_noise luen_core luen_init
idx  = 8;
out7 = sim('QuadVisionControl_noise_luen');
```

> **One at a time.** `observers\dkf\` and `observers\dkf+noise\` both contain `dkf_core.m` and `dkf_init.m`; the same is true of the two Luenberger folders. The files are identical, so a clash is harmless — but the `params` files are not identical, and those are what change the result. Keep one of each pair on the path.

4.8 Everything at once

To run the eight comparison models back to back and leave every result ready to plot:

```
addpath(fullfile(ROOT,'observers','dkf'))
addpath(fullfile(ROOT,'observers','luenberger'))
addpath(fullfile(ROOT,'observers','lstm'))
scen = 8;          % the scenario, same meaning as idx
run_all_sims
```

It fills `out1`…`out8` in one pass and prints a summary line for each. A model it cannot find is skipped with a warning rather than stopping the run. It does not include the Neural Luenberger; run that one separately as in 4.6. Note it uses the variable `scen`, not `idx`.

5. Looking at the results

Every plotting script works the same way: it asks for a **run number**, fetches the matching `outN` from the workspace, and draws it. You never pass arguments — you edit one line near the top of the script, or set the variable before running it.

Run number	Variable	Condition
1	out1	No delay — the reference
2	out2	160 ms delay, uncompensated
3	out3	Delay + Delayed Kalman Filter
4	out4	Delay + Delayed Luenberger observer
5	out5	Delay + LSTM residual predictor
6	out6	Delay + DKF, noisy camera
7	out7	Delay + Luenberger, noisy camera
8	out8	Delay only, noisy camera — the noisy baseline
10	out10	Delay + Neural Luenberger Observer
0	—	Whatever the last simulation left loose in the workspace

5.1 A single run

To see the DKF result after running 4.3, open `plots\plot_distance_error.m`, find the line marked `<<<<<< EDIT THIS LINE` and set it to 3:

```
RUN_ID = 3;          % 3 = DKF.  Then press Run in the editor.
```

Or, without opening the file at all, from the Command Window:

```
RUN_ID_FORCE = 3;    % 3 = DKF
plot_distance_error
clear RUN_ID_FORCE
```

It draws the separation from the target and the error about the 2 m setpoint, and prints the settling time, final error, minimum separation and RMSE — the numbers in the results tables. Give it a list instead of a number, `RUN_ID = [1 2 3 4 5]`, and it opens one labelled window per condition.

5.2 Several runs on one figure

The three `compare_*` scripts take a list on their `IDS` line instead:

```
IDS = [1 2 3 4 5];   % noise-free comparison
IDS = [8 6 7];       % noise campaign
```

5.3 What each script draws

Script	What it gives you
`plot_distance_error.m`	Separation and the error about the 2 m stand-off, two stacked panels, plus the printed settling time, final error, minimum separation and RMSE. **The main results figure.**
`compare_distance.m`	Separation for several conditions on one axis, with a comparison table. The headline figure of the comparison chapter.
`compare_trajectory.m`	The 3-D flight paths of several conditions against the same target path.
`compare_z_saturation.m`	The altitude channel for several conditions — where the delay shows up first and most violently.
`plot_z_saturation.m`	The same signal for a single run, against the vertical-speed limit.
`visualize_3d_trajectory_spotlight.m`	Animated 3-D pursuit with playback controls and a floating distance read-out. The best single picture of a run.
`visualize_3d_trajectory_old.m`	The same animation without the distance label.
`visualize_camera.m`	What the camera sees: the target blob on the image plane, with playback.
`visualize_combined.m`	The 3-D view and the camera view side by side, synchronised.

Helpers — never run these directly

File	Role
`run_all_sims.m`	Runs the eight comparison models in sequence and fills out1…out8.
`pickrun.m`	Turns a run number into the right `outN` object.
`getsig.m`	Reads one logged signal out of a run, whatever format it was logged in.
`loadruns.m`	Collects several runs at once for the `compare_*` scripts.
`runmany.m`	Re-runs a plotting script once per run number when given a list.

6. What every file does

6.1 Delayed Kalman Filter — `observers\dkf\`

File	Role
`dkf_core.m`	The filter. Predict at 200 Hz on the vehicle's own acceleration, correct the oldest slot when a frame lands, replay the correction forward to the present. Every equation of the method is here.
`dkf_params.m`	All constants and gains: the delay D, the window size, σ_a, σ_{uv}, σ_s, the innovation gate, the camera intrinsics. The only file to edit to change tuning.
`dkf_init.m`	Allocates the six-state vector, its covariance and the 33-slot delay window.
`dkf_block.m`	The Simulink wrapper: holds the state between steps and calls `dkf_core`. Its text is already pasted inside the model; the file is here so it can be read.

6.2 DKF under noise — `observers\dkf+noise\`

Identical filter, different trust in the sensor. `dkf_params_noise.m` states the noise actually injected (σ_{uv} = 1.5 px, σ_s = 0.25 px) and re-sweeps only σ_a, to [2 2 4]. `dkf_block_noise.m` differs from `dkf_block.m` by one line: which params file it calls. `dkf_core.m` and `dkf_init.m` are the same files, copied here so the folder runs on its own.

6.3 Luenberger observer — `observers\luenberger\`

File	Role
`luen_core.m`	The observer: same window and replay as the DKF, but it inverts the pixels into a position and corrects there with two fixed gains.
`luen_params.m`	ω_n, ζ and the camera period, and the gains L_p, L_v computed from them by pole placement.
`luen_init.m`	Allocates the state and the 33-slot window. No covariance exists here.
`luen_block.m`	The Simulink wrapper, already pasted into the model.

`observers\luenberger+noise\` is the same four files with `luen_params_noise.m` in place of `luen_params.m`: ω_n drops from 8 to 4, because the gain had to be halved once the camera became noisy.

6.4 LSTM residual predictor — `observers\lstm\`

File	Role
`lstm_block.m`	Builds the 20 input features from the delayed measurement and the vehicle's own states, calls the network, and adds the predicted residual back onto the measurement. **Called from the model at run time.**
`lstm_forward_core.m`	One step of the network by hand: the four gates, the cell and hidden state, the linear output. Loads the weights on its first call.
`lstm_weights.mat`	The trained network: W_{ih}, W_{hh}, the biases, the output layer, and the feature mean and standard deviation used to standardise the inputs. This copy is the round-4 filter-aware network.
`lstm_config.m`	The single definition of the 20 features. Not needed to run — included so the feature list can be read.
`smooth_block.m`	The output low-pass, τ = 0.02 s. Already inside the model; the file is here to be read.

6.5 Neural Luenberger Observer — `observers\neural luenberger\`

File	Role
`velocity_net_block.m`	Same 20 features as the LSTM, but the network's three outputs are image-plane velocities rather than a position correction.
`obs_integrator_block.m`	The observer: integrates that velocity, and corrects the drift with the delayed innovation applied as L/N_{cam} at every step. Holds L = 0.10, the 600 px drift leash and the 1 px radius clamp.
`lstm_forward_core.m`	The same network forward pass as the LSTM method, loading the weights beside it.
`lstm_weights.mat`	The trained velocity network — round 2. A different network from the one in `observers\lstm\`, under the same file name.

7. If something goes wrong

Message or symptom	Cause and fix
Undefined function or variable `'idx'`	The scenario was not set. Type `idx = 8;` and run again.
Undefined function or variable `'IC_pos'`	Only the LSTM and Neural Luenberger models need these. Paste the four IC lines from section 3.2.
Unable to resolve the name `dkf_params` (or luen_params, lstm_forward_core …)	That method's folder is not on the path. Run the `addpath` line at the top of its recipe.
The LSTM and the Neural Luenberger give the same trajectory, or a strange one	Both observer folders are on the path and the wrong `lstm_weights.mat` is being loaded. `rmpath` the one you are not using and re-run.
A second run differs from the first, or starts oddly	Network memory carried over. Run the `clear` line from the recipe before each simulation.
Strange code-generation errors after moving the folder	Stale build cache. Close the model, delete any `slprj` folder and `.slxc` files beside the models, reopen. They rebuild themselves.
A plotting script says a run is missing	That condition has not been simulated yet in this MATLAB session. Run it, or set the run number to one you have.

A quick end-to-end check. Paste this and you should get two figures: the delay-free reference and the DKF, on the hardest scenario.

```
ROOT = 'C:\Users\Abbass\Desktop\New folder';
cd(ROOT); addpath(fullfile(ROOT,'plots'), fullfile(ROOT,'observers','dkf'))
idx = 8;
out1 = sim('QuadVisionControl');
out3 = sim('QuadVisionControl_dkf');
RUN_ID_FORCE = 1; plot_distance_error
RUN_ID_FORCE = 3; plot_distance_error
clear RUN_ID_FORCE
```